SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

Somaiya TRUST

StripeSense
Urine Strip Colour Analysis

Detailed Project Report

Submitted by
Yashvardhan Samadhiya 16010124058
Amey Salunkhe 16010124057
Shrived Deshmukh 16010124071

B.Tech Computer Engineering
Third Year / Semester V

Faculty: Shruti Javakar
Academic Year: 2026-2027

Mini Project: FSD-MERN Stack

GitHub Repository: N.A.
Local implementation completed; repository publication pending.

Table of contents

1. Introduction 5
2. Requirements and Technology Stack 6
3. System Design 7
4. Application Design and Implementation 10
5. Testing and Results 13
6. GitHub and Deployment 14
7. Challenges and Individual Contributions 15
8. Conclusion and Future Scope 16
References 17

List of figures

Figure 1 System architecture 7
Figure 2 Application workflow 8
Figure 3 Database relationships 8
Figure 4 Public landing page 10
Figure 5 Sign in screen 11

List of tables

Table 1 Functional requirements 6
Table 2 Non functional requirements 6
Table 3 Technology stack 6
Table 4 Modules and workflow 7
Table 5 Database collections 8
Table 6 Implemented API endpoints 9
Table 7 CRUD coverage 12
Table 8 Executed test cases 13
Table 9 Deployment details 14
Table 10 Challenges and solutions 15
Table 11 Individual contributions 15

1. Introduction

1.1 Background

Urine reagent strips use colour changes on multiple pads to support screening. A photographed strip can be compared with a reference chart, but the image and its patient details must also be recorded consistently. A web application provides a convenient way to collect an image, review colour matches and revisit earlier results.

StripeSense was developed as a Full Stack Development mini project. The main academic objective is to integrate React, Express, Node.js and MongoDB through a usable application. Its implemented image pipeline uses reference-colour comparison rather than a trained machine-learning model. The current outputs are unvalidated estimates and do not diagnose a medical condition.

1.2 Problem Statement

Individuals and doctors need an organised way to associate strip images and colour-matching results with the correct person. Keeping photos and notes separately makes retrieval difficult and creates the risk of mixing records. The project addresses this software workflow through account-based patient records, authenticated uploads and searchable saved reports. It does not address clinical validation or replace a laboratory analyser.

1.3 Proposed Solution

The application provides a public landing page and a private workspace. A user registers as a doctor or individual, signs in, adds a patient, uploads a horizontal ten-pad photo and a reference JSON profile, selects the pad row and submits the analysis. Express validates the request, runs the colour comparison and stores the result with its source image in MongoDB. React retrieves the report through Axios for review, history filtering and printing.

1.4 Objectives and Scope

- Provide registration, sign in, persistent sessions and sign out.
- Create patient records and associate each test with its patient and account owner.
- Process a ten-pad image using a supplied colour-reference profile.
- Save and retrieve reports, source images and analysis metadata in MongoDB.
- Demonstrate API integration, input validation, account isolation and automated testing.

Scope includes a desktop-first interface, light and dark themes, patient creation, reference matching, history, report printing and test deletion. Hardware capture, trained ML, verified concentration estimation, medical diagnosis, shared clinical teams and public cloud deployment are outside the implemented scope. Patient/test editing is a future enhancement.

2. REQUIREMENTS AND TECHNOLOGY STACK

2.1 Users and Functional Requirements

Table 1 Functional requirements

ID	User or function	Implemented behavior
FR1	Account access	Register, sign in/out and persist a private session.
FR2	Patient directory	Add named patients with optional auto-generated IDs.
FR3	Photo analysis	Upload a ten-pad photo/profile, crop and compare colours.
FR4	History and reports	Retrieve account-owned tests, source images and printable reports.
FR5	Privacy and deletion	Block other-account access and delete an owned test.

Doctor and individual accounts use the same private data model; account type changes wording, not access to other users' records.

2.2 Non-Functional Requirements

Table 2 Non functional requirements

Category	Requirement
Usability	Clear forms, desktop navigation, readable errors and persistent light/dark mode.
Performance	Bounded uploads/pixels and at most 500 rows per list request. No latency claim.
Reliability	Save complete results and reject invalid images/profiles before insertion.
Security	Password hashing, expiring sessions and owner-filtered queries.
Maintainability	Separate npm workspaces, reusable components and consistent Prettier formatting.

The desktop interface provides clear forms and errors. Upload limits bound processing work; estimates remain separate from medical interpretation.

2.3 MERN Technology Stack

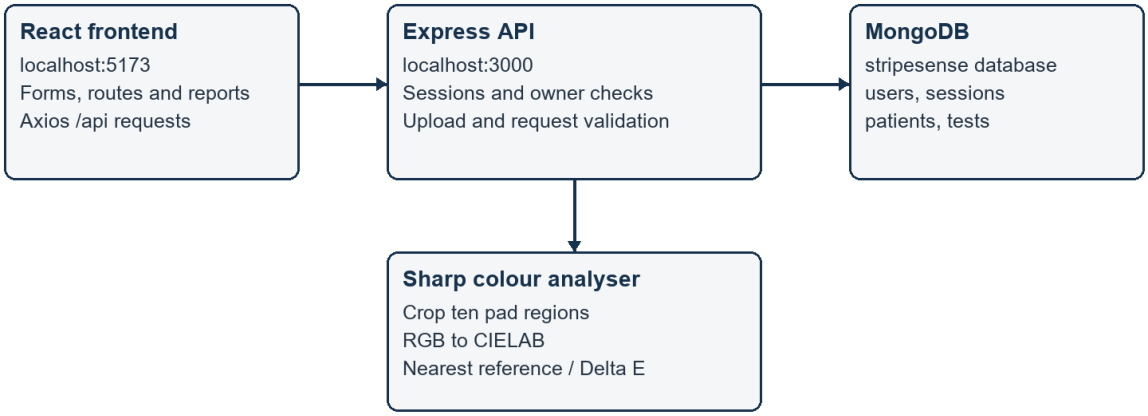
Table 3 Technology stack

Layer	Technology	Purpose
Frontend	React 19, React Router, Vite	Components, routes and development/build tooling.
HTTP client	Axios	JSON and multipart requests to /api.
Backend	Node.js, Express 5, Multer	API, authentication and uploads.
Database	MongoDB, official Node.js driver	Persistent documents, references and indexes; no Mongoose.
Images	Sharp, CIELAB	Decode and compare sampled pad colours.
Verification	node:test, Vite build	Unit/integration tests; browser UI checks.
Deployment	Local workspaces	Local frontend/API; no cloud hosting.

Separate npm workspaces organise the frontend and backend. MongoDB uses the official Node.js driver, not Mongoose.

3. SYSTEM DESIGN

3.1 System Architecture



Axios requests reach Express through the Vite proxy. The backend validates ownership, calls the analyser and persists the result. Responses follow the same API path back to React.

Figure 1 System architecture

3.2 Modules and Workflow

Table 4 Modules and workflow

Module	Main functions
Authentication	Register/login/logout, current user, session cookies and expiry.
Patient records	Create/list owned patients and auto-generate IDs.
Colour analysis	Image/profile validation, crop sampling and nearest-reference matching.
Reporting and history	Save, filter, view, print and delete owned tests.

A test is processed only after a valid account session and owned patient have been selected. The image and reference profile travel together in a multipart request. After saving, the report can be retrieved independently of the upload page, so refreshing the browser does not discard the result.

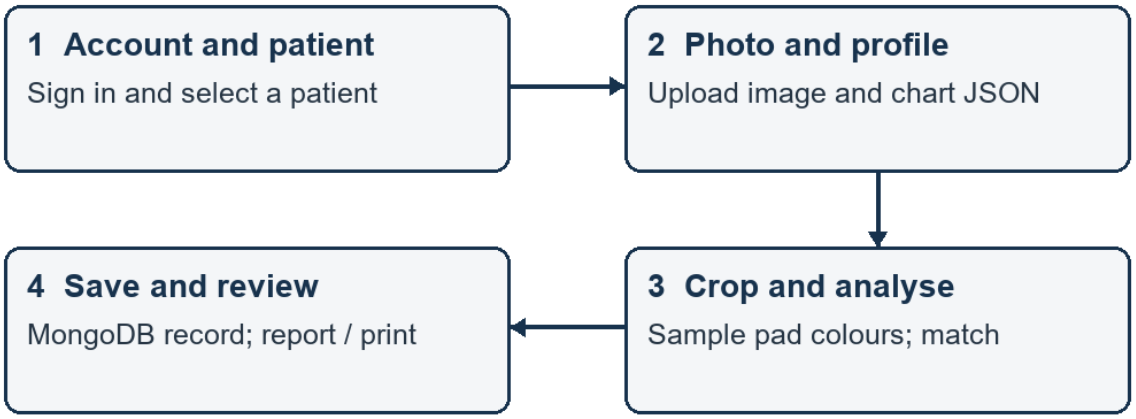


Figure 2 Application workflow

3.3 Database Design

The users, sessions, patients and tests collections use ObjectId references for ownership and history. Each saved test includes a patient snapshot and its source image as BSON binary.

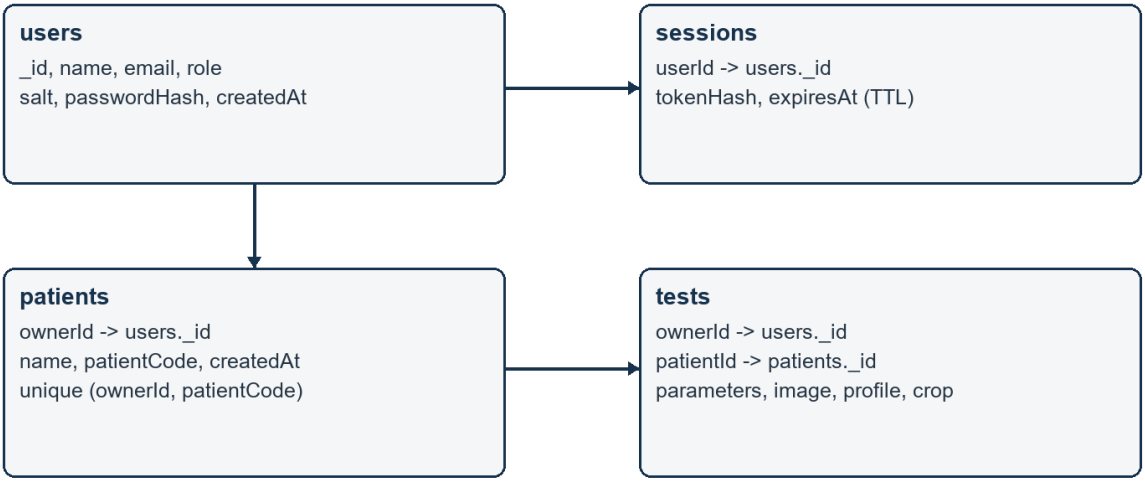


Figure 3 Database relationships

Table 5 Database collections

Collection	Important fields	Purpose
users	_id, name, email, role, salt, passwordHash, createdAt	Account identity and password verification.
sessions	tokenHash, userId, expiresAt	Expiring account sessions.
patients	_id, ownerId, name, patientCode, createdAt	Private patient directory.
tests	ownerId, patientId, createdAt, parameters, image, crop, referenceProfile	Saved report, image and reproducibility metadata.

Email is unique across users. Patient codes are unique within an account. Session token hashes are indexed, and expiresAt has a TTL index. Session expiry is also checked at request time. Test queries include ownerId to enforce isolation. Records created before ownership was introduced are preserved but not assigned automatically.

3.4 API Design

Table 6 Implemented API endpoints

Method	Endpoint	Purpose
POST	/api/auth/register	Create account and session.
POST	/api/auth/login	Authenticate and issue session cookie.
GET	/api/auth/me	Retrieve current account.
POST	/api/auth/logout	Revoke current session.
GET	/api/health	Check database availability.
GET	/api/reference-example	Retrieve sample profile; sign in required.
GET, POST	/api/patients	List or create owned patients.
GET	/api/tests?patientId=...	List/filter owned tests.
POST	/api/tests	Upload, analyse and save a test.
GET	/api/tests/:id	Retrieve an owned report.
GET	/api/tests/:id/image	Serve the saved PNG.
DELETE	/api/tests/:id	Delete owned test and embedded image.

Authentication and data APIs use JSON except POST /api/tests, which accepts multipart form data. Successful creation returns HTTP 201; unauthenticated access returns 401; unknown or inaccessible resources return 404. The API exposes no patient/test PUT or PATCH endpoint in the current version.

4. APPLICATION DESIGN AND IMPLEMENTATION

4.1 User Interface and Navigation

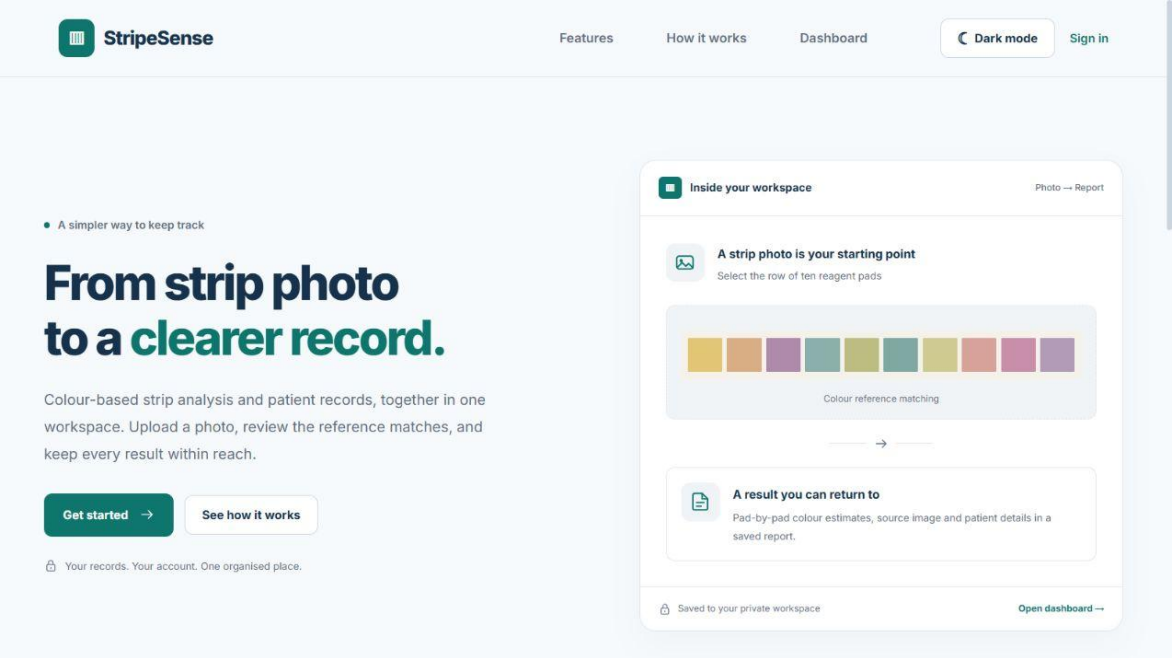


Figure 4 Public landing page

The public landing page explains the implemented workflow and links to sign in. The authenticated workspace provides Dashboard, Analyse photo, Test history and Patients navigation. Sign in includes registration and account-type selection. Light/dark mode is available on public and private screens and is remembered in localStorage.

The dashboard displays saved record counts and recent tests rather than illustrative graphs. Analysis includes patient selection, profile/image upload and a crop overlay. Reports show colour categories, measured/reference RGB, Delta E and review flags. The layout prioritises desktop use; the private workspace has a minimum width and is not presented as fully mobile-optimised.

4.2 Frontend Implementation

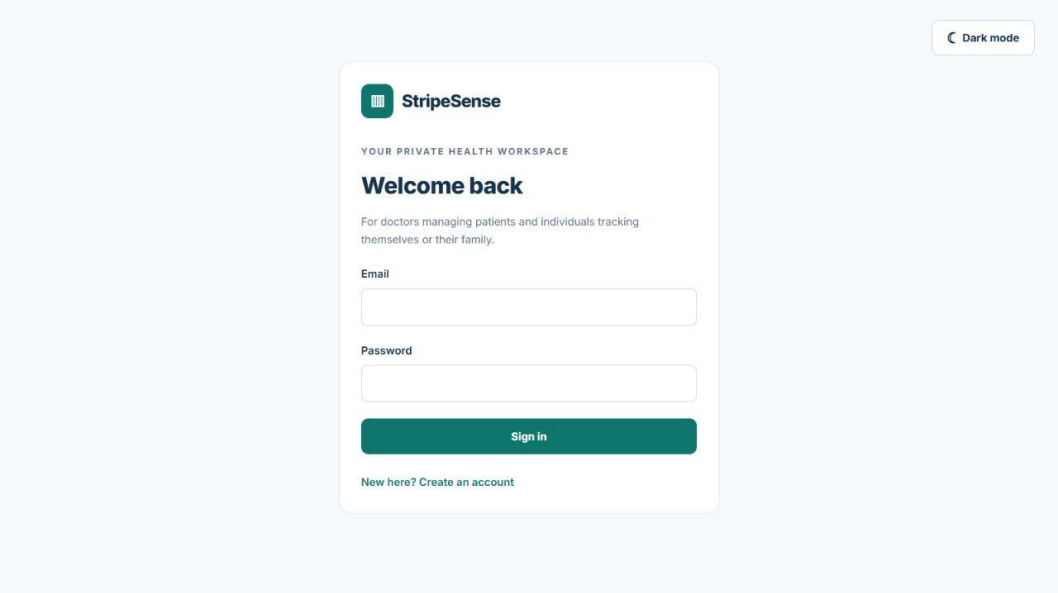


Figure 5 Sign in screen

React functional components use useState, useEffect and useRef for forms, asynchronous records and image selection. React Router separates public pages from private routes. AuthProvider retrieves the current session; RequireAuth redirects signed-out users and displays connection errors without exposing protected pages.

A shared Axios client uses /api with a request timeout. FormData carries the photo, patientId, profile and crop. The image preview uses object URLs and releases them when replaced. ThemeProvider persists the selected appearance. Components for reports and interpretation keep detailed colour measurements separate from the user-facing explanation.

4.3 Backend and API Implementation

Express applies a JSON size limit, route-scoped Multer upload handling and central error responses. Multer accepts JPEG, PNG or WebP metadata with an 8 MB file limit; Sharp performs actual decoding and limits the decoded pixel count. A normalized image above 10 MB is rejected before persistence.

Passwords are salted and hashed with scrypt. Login issues a random session token in an HttpOnly, SameSite=Strict cookie; only its SHA-256 hash is stored in MongoDB. Logout deletes the session. Origin checks and in-process attempt limits provide additional protection. Production cookies require HTTPS; deployment hardening remains necessary before real-world use.

4.4 Database and CRUD Implementation

Table 7 CRUD coverage

CRUD	Frontend action	Backend and database action
Create	Add patient; submit analysis	POST endpoints; insertOne into patients/tests.
Read	Directory, dashboard, history, report	GET endpoints; find/findOne with ownerId.
Update	Not implemented for patient/test records	No PUT/PATCH route; planned enhancement.
Delete	Confirm deletion on saved report	DELETE owned test; deleteOne removes embedded image too.

The MongoDB driver performs insertOne, find, findOne and deleteOne operations. Patient creation stores the account owner and generates a patient code when omitted. Uploading a test verifies that the selected patient belongs to the signed-in account, then inserts a complete analysis record. List queries omit binary images; a separate authenticated endpoint serves the saved PNG.

Read operations support patient-filtered test history, individual reports and source-image retrieval. Deleting a test removes its embedded source image with the record. Patient and test editing are not implemented. Session creation/deletion belongs to authentication and is not a substitute for a record-editing feature.

4.5 Validation and Error Handling

Missing patient names and unsupported uploads are rejected with actionable errors. Duplicate patient codes within an account return 409. Malformed profiles, invalid crop rectangles and image decode failures return an analysis validation error rather than storing a partial test.

Unknown or other-account report IDs return 404, including image and deletion endpoints. Network failures produce a connection message in the frontend. Near-tied colour matches are flagged for review and are not forced into a known estimate category.

5. TESTING AND RESULTS

5.1 Testing Approach

The Node.js test runner executes analyser and interpretation unit tests plus a real Express/MongoDB integration test. The integration test uses an ephemeral HTTP port and a uniquely named temporary database, which it removes after execution.

On 30 October 2026, all eight tests passed with TEST_MONGODB_URI set to local MongoDB. The frontend production build passed. Browser checks covered navigation, protected-route redirects and theme persistence. Postman was not used for these recorded tests.

5.2 Test Cases

Table 8 Executed test cases

ID	Executed check	Expected result	Observed result	Status
TC01	Known pad colours	Correct match	All ten match	PASS
TC02	Invalid crop/image/profile	Reject invalid input	Rejected	PASS
TC03	Lab conversion	Expected lightness	Assertions pass	PASS
TC04	Auth and storage	Persist / isolate	Checks pass	PASS
TC05	Presentation labels	Map display text only	Stored labels preserved	PASS
TC06	Estimate flags	Correct priority	Checks pass	PASS
TC07	Unknown labels	No classification	Unknown returned	PASS
TC08	Near-tied colours	Review only	Review group used	PASS

These checks were executed with generated inputs and temporary records. They verify software behavior, not diagnostic accuracy.

5.3 API and Database Verification

The integration test verifies authenticated patient creation, image upload, report/image retrieval, deletion and account isolation. Database assertions confirm hashed password storage.

Local verification uses mongodb://127.0.0.1:27017 and the stripesense database. Compass can inspect development records; tests use a separate temporary database.

5.4 Final Results

Account, patient, upload, reporting and deletion workflows pass automated checks. Outputs remain unvalidated colour estimates. Record editing, public deployment and clinical validation are pending.

6. GITHUB AND DEPLOYMENT

6.1 GitHub Repository

A GitHub repository has not yet been created for the project. Repository URL and hosted source-control history are therefore N.A. The local directory structure separates frontend and backend code as shown below. Publishing the source and establishing team branches/reviews are planned next steps.

```
stripe-sense-vite/
  frontend/
    src/  pages, auth, theme, components, api, utils
    index.html, vite.config.js, package.json
  backend/
    app.js, auth.js, db.js, index.js, analyzer.js
    *.test.js, scripts/, .env.example
  samples/  generated images and reference JSON
  package.json, package-lock.json, README.md
```

Local project structure

6.2 Deployment Details

Table 9 Deployment details

Item	Details
GitHub repository	N.A. - publication pending.
Frontend URL	http://localhost:5173 (local only)
Backend URL	http://localhost:3000 (local only)
Database	Local MongoDB; database stripesense
Deployment platform	N.A. - not publicly deployed.

The application currently runs locally. npm run dev starts Vite and Express together. The frontend is available at http://localhost:5173 and the backend at http://localhost:3000. Vite forwards /api requests to Express. MongoDB runs locally; no public frontend/backend URL or cloud platform is assigned.

7. CHALLENGES AND INDIVIDUAL CONTRIBUTIONS

7.1 Challenges and Solutions

Table 10 Challenges and solutions

Challenge	Solution	Outcome
Pad sampling initially read the wrong region	Materialise each Sharp crop before computing image statistics.	Known-colour tests pass for all ten pads.
Disconnected frontend/API processes	Use coordinated npm workspace scripts and a Vite /api proxy.	Both services start from npm run dev.
Private records needed account isolation	Session authentication plus ownerId in all protected queries.	Other-account report/image/delete requests are blocked.
Colour distance mistaken for medical severity	Separate category estimates, Delta E and consultation disclaimer.	Reports avoid unsupported diagnosis or urgency claims.

The principal implementation challenges were establishing a complete upload-to-history flow, correctly sampling each pad, protecting patient-linked records and avoiding misleading medical claims. The project resolved these through a shared API contract, verified crop sampling, authenticated ownership filters and a clear distinction between colour estimates and diagnosis.

7.2 Individual Contributions

Table 11 Individual contributions

Student	Frontend	Backend and API	Database	Supporting work
Yashvardhan Samadhiya 16010124058	Not primary focus	Primary focus	MongoDB integration	Supporting Python scripting
Amey Salunkhe 16010124057	Primary focus	Supporting work	Not separately assigned	UI development
Shrived Deshmukh 16010124071	Primary focus	Supporting work	Not separately assigned	UI development

The distribution below records the contributions supplied by the team. Amey Salunkhe and Shrived Deshmukh focused on the frontend with supporting backend work. Yashvardhan Samadhiya focused on backend services, MongoDB integration and supporting Python scripting. The application's deployed analysis path itself is JavaScript-based; no Python inference service is required.

8. CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

StripeSense brings account-based patient records, reference-colour matching and saved reporting into one MERN application. React provides the interface, Axios connects it to Express, and MongoDB persists users, sessions, patients and tests. Automated checks confirm the implemented software behavior, including ownership isolation and retrieval of saved results.

The project demonstrates full-stack integration without claiming clinical measurement accuracy. Its colour outputs are estimates from a supplied reference profile. Further work is required for complete record editing, public deployment, manufacturer calibration and any validated medical interpretation.

8.2 Future Scope

Future work includes patient/test editing with audit history; password reset and email verification; pagination beyond the current 500-record lists; broader mobile workspace support; private object storage for larger images; and secure public deployment. Manufacturer-specific reference profiles, varied-layout pad detection and evaluation against labelled real-strip images would be required before extending analysis beyond the current controlled scope. Shared-care access would need explicit consent and fine-grained permissions.

References

[1] React, React documentation. https://react.dev/learn

[2] Express, Express documentation. https://expressjs.com/

[3] MongoDB, Node.js driver documentation. https://www.mongodb.com/docs/drivers/node/current/

[4] Axios, Axios documentation. https://axios-http.com/docs/intro

[5] Sharp, Image processing documentation. https://sharp.pixelplumbing.com/

[6] Node.js, Test runner documentation. https://nodejs.org/api/test.html

[7] DiaSys, UDStrip-12 product information and instructions. https://www.diasys-diagnostics.com/products/dedicated-instruments/respons-uds100/udstrip-12/

[8] MedlinePlus, Urinalysis. https://medlineplus.gov/ency/article/003579.htm